ONE-PAGE BRIEF

Post-quantum cryptography review: rust-secp256k1 @ fe8b3bec962f

Client: rust-secp256k1 — public repository (passive review) · Source: github.com/rust-bitcoin/rust-secp256k1, commit `fe8b3bec962f` · Reviewed: 2026-10-29 19:56 UTC · Risk profile: Crypto exchange, custodian or wallet

POST-QUANTUM MATURITY

Level 0 of 4

no post-quantum cryptography

FINDINGS

0 Critical **0** High **3** Medium

0 Low **0** Info

QUANTUM TIMELINE (MOSCA: X + Y − Z)

Signatures: 10 + 5 − 8 = 7 → **too late**

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

WHAT MATTERS MOST

Medium QSI-2026-06682	**Quantum-vulnerable ECDSA: transaction and message signing (23 file(s))** preliminary What to do: Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains...
Medium QSI-2026-06683	**Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (2 file(s))** preliminary What to do: Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains...
Medium QSI-2026-06684	**Quantum-vulnerable ECDSA: role not determined (2 file(s))** preliminary What to do: Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form...

WHAT THE QUANTUM-VULNERABLE CRYPTOGRAPHY PROTECTS

Role	Algorithms	What to do
Medium transaction and message signing	ECDSA, Schnorr (BIP-340)	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum...
Medium role not determined	ECDSA	Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205)...

Static review of source code and manifests; files analysed: 118. No requests were sent to the owner's systems.

Full report, content SHA-256: `0383ea7ca33a559a1937e9fb54875aaf90e0e5036351bd80f34861433622d557`

Full report signature (report.sig.json): Ed25519 key, SHA-256 fingerprint `5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`. bundle.sig.json lists the SHA-256 of every file in this bundle and is signed too; verify-bundle checks it offline.

Preliminary findings need confirmation in the code; the full report gives the evidence, the limits of each check and how to re-test.